Margem: technical feature map

Algorithmic Working Capital & Margin Optimization

Causal Double-ML dynamic pricing and liquidity engine on Olist data

24 October 2026

Every technical feature in the project, what it is for, where the code lives and where it shows up on the website (`operator-x.github.io/margem`). Code paths are relative to the repository root.

Phase 1: Data and unit economics

Feature	What it is used for	Code	On the site
Weekly product sales table	Turns raw Olist orders into weekly sales per product (118 products)	`engine/data.py`	Feeds everything
Profit per sale (CM2)	Profit after cost, 10% fee and freight; the optimisation target, never revenue	`engine/finance.py`	Products page, product pop-up
Assumed product costs	Olist publishes no costs, so each category gets a fixed share of price	`engine/data.py`, `engine/config.py`	All profit numbers
SELIC interest-rate feed	Pulls Brazil's official interest rate from the central bank API	`engine/finance.py`	Overview header, Scenario lab
Holding cost (SELIC + 6%)	What it costs to keep stock sitting unsold	`engine/finance.py`	Product pop-up, Scenario lab
Days of stock, safety stock	How long stock lasts and how much buffer is needed	`engine/finance.py`	Products page

Phase 2: Forecasting

Feature	What it is used for	Code	On the site
LightGBM forecast (Tweedie loss)	Predicts next week's sales; handles the 52% of weeks with zero sales	`engine/forecast.py`	Product → Forecast
Seasonal naive and SARIMAX	Simple benchmarks to beat	`engine/forecast.py`	Evidence → forecast pop-up
Walk-forward testing	Tests forecasts without ever peeking at the future	`engine/forecast.py`	Evidence
Leakage fix (last week's price)	Stops the model using information it would not have had	`engine/forecast.py`	Covered by tests
Money-based scoring	Turns forecast error into stockout losses and cash tied up	`engine/forecast.py`	Evidence

Phase 3: Causal price sensitivity

Feature	What it is used for	Code	On the site
Double ML	Measures how price really changes demand, removing coincidences such as Black Friday	`engine/elasticity.py`	Evidence page (main chart)
Cross-fitting in time blocks	Keeps the ML models from leaking future data into the estimate	`engine/elasticity.py`	Evidence → statistics
Product fixed effects, clustered standard errors	Compares each product with itself; honest error bars	`engine/elasticity.py`	Evidence (error bars)
Naive and pooled regression	Shows what goes wrong without causal methods (about 3× overstated)	`engine/elasticity.py`	Evidence headline "3×"
DerSimonian–Laird shrinkage	Stabilises noisy per-product estimates by pulling them toward the category average	`engine/elasticity.py`	Product pop-up
Corrections (log fix, sign restriction, cap at −5)	Keeps estimates economically sensible	`engine/elasticity.py`	Product pop-up chips
Robustness check (weeks with sales)	Confirms the result holds under a second definition	`engine/elasticity.py`	Evidence → statistics

Phase 4: Price optimiser

Feature	What it is used for	Code	On the site
Profit-maximising price search	Finds the best price (401-point grid, then Brent refinement)	`engine/optimize.py`	Overview cards, product pop-up
Price floor (1.15× cost; 1.00× if stock is old)	Never sells below cost; allows clearance of stock older than 8 weeks of supply	`engine/optimize.py`	Product pop-up chips
±15% price band	Never shocks buyers with large jumps	`engine/optimize.py`	Overview card text
Margin vs cash-recovery modes	Optimise this week's profit, or clear stock to free cash	`engine/optimize.py`	Scenario lab goal switch
Uncertainty band	Re-solves the price under 200 draws of price sensitivity	`engine/optimize.py`	Product → Simulate
Lerner discount test	Checks whether any discount could add profit (none of 72 do)	`engine/finance.py`	Evidence, Overview

Phase 5A: AI season planner (reinforcement learning)

Feature	What it is used for	Code	On the site
26-week season simulator	Sells one lot of stock; leftovers go to an outlet at half of cost	`engine/rl.py`	Season planner
Bayesian demand learning	The planner updates its estimate of price sensitivity from its own sales	`engine/rl.py`	Season replay

Feature	What it is used for	Code	On the site
PPO agent	The AI that decides each week whether to hold the price or start clearing	`engine/rl.py`	Season replay ("AI season planner")
Exact plan and oracle (dynamic programming)	Benchmarks: the best possible plan, and one that knows true demand	`engine/rl.py`	Season replay → More options
Value of information	What knowing true demand would be worth	`engine/rl.py`	"Value of knowing the truth" note
Out-of-sample scorecard	Results over 30 unseen seasons per product	`engine/rl.py`	Bottom chart of the Season planner

Phase 5B: GenAI forecasting

Feature	What it is used for	Code	On the site
Chronos-Bolt and Chronos-2	Amazon foundation models that forecast with no training on our data	`engine/foundation.py`	Evidence → forecast pop-up
LightGBM quantile forecasts	Our trained model's low / median / high forecasts	`engine/foundation.py`	Product → Forecast band
Newsvendor stocking backtest	Compares models by stockouts and cash tied up in stock	`engine/foundation.py`	Evidence → forecast pop-up

Phase 5C: LLM customer voice

Feature	What it is used for	Code	On the site
Claude review labelling	Scores 3,351 reviews for sentiment, quality, late delivery and value	`engine/reviews.py`	Product → What buyers say
Reviews as Double-ML controls	Makes the price estimate more accurate ($-0.34 \to -0.45$)	`engine/elasticity.py`, `engine/pipeline.py`	Evidence → voice card
Validation against star ratings	Shows the labels are trustworthy (87.7% agreement)	`engine/reviews.py`	Evidence → voice card
Moderation test	Asks whether reviews change how price-sensitive buyers are	`engine/elasticity.py`	Voice card → details

Phase 6: Margem web app

Feature	What it is used for	Code	On the site
Five-page app	Overview, Products, Season planner, Evidence, Scenario lab	`web/template.html`	Whole site
Capi mascot and hand-drawn style	A memorable identity	`web/template.html`, `brand/`	Everywhere
Expanding cards, scroll animations	"Simple first, details on request"	`web/template.html`	Everywhere
Cmd+K product search	Jump to any product quickly	`web/template.html`	Press Cmd+K

Feature	What it is used for	Code	On the site
Scenario lab	Re-prices all 72 products live as interest rates, stock and fees change	`web/template.html`	Scenario lab
In-browser engine	Optimiser and AI planner ported to JavaScript, so everything runs on a phone	`web/template.html` (ENGINE block)	Simulate, Season replay, Scenario lab
Data export	Packs all results into the page	`engine/webexport.py`	Not visible
Optional Python server	The same page served by FastAPI, with API endpoints	`engine/api.py`	Not used by the public site
GitHub Pages hosting	Public website with no server	`web/publish.sh`	Public website
QR codes and Capi images	Poster assets	`brand/make_qr.py`, `brand/render_capi.js`	Poster

Running everything and quality

Feature	What it is used for	Code
One-command pipeline	Runs Phases 1–6 and writes the results report	`engine/pipeline.py`
Central settings (seed 42, fees, floors)	One place for every rule; reproducible results	`engine/config.py`
Results report	All result tables in one file	`reports/results.md`
40 automated tests	Checks guardrails, data leakage, the AI benchmarks, and that the browser matches Python	`tests/`
Separate processes for PyTorch and LightGBM	Prevents a crash when both load on macOS	`engine/foundation.py`, `engine/webexport.py`